

An Agentic AI Workflow for Parameter Estimation of Complex Differential Equation Systems

Saakaar Bhatnagar¹

¹*<Affiliation, City, State/Country>*
ORCID: <ORCID ID>

Corresponding author: Saakaar Bhatnagar, <email address>

Abstract

Parameter estimation for ordinary differential equation (ODE) models fitted to experimental data is an iterative process often made difficult by heuristic fitting methods that make severely limiting assumptions, noisy experimental data, large search spaces and stiff ODE dynamics. Posing the estimation problem as an optimization problem is a commonly used method, but usually needs advanced knowledge of optimization algorithms and is prone to implementation errors and inefficiencies. This chapter describes an AI agent centered workflow for converting a lightweight estimation problem specification into a reproducible calibration pipeline. The overall task of estimation is broken down into sub steps, with the human in the loop confirming the agent's work. The agent's context for each step is populated with sub task-specific knowledge, enabling expert execution instead of random guesswork. The user provides key definitions in a structured file system. The workflow emphasizes reproducibility and audit through inspection of generated code, and diagnostic checks that help distinguish numerical, modeling, and data-quality failures. Overall, this workflow allows easy and robust iteration on model form, search space and objective functions, which critically affects the obtained models.

Key words

Parameter Estimation; Ordinary Differential Equations; Global Optimization; Differentiable Programming; Gradient-Based Optimization

1 Introduction

1.1 Motivation

add shaowu and nithin cites

Ordinary differential equation (ODE) models are widely used to represent biological, chemical, and physical systems, these often are put into bigger models, eg battery

A common strategy is to frame the fitting problem as an optimization problem, by minimizing the solution of the ODE or some derivative of it to the experimental data. **include citations**. Usually, practitioners employ global search methods, where the search is driven by population methods such as swarm optimization, differential evolution or genetic algorithms. These methods are agnostic to the physics of the problem, are trivially parallelizable and are robust to failure of samples.

These methods return a good initial guess to the problem, which can be fine-tuned using gradient based methods to obtain a more accurate fit. An additional challenge for gradient based methods is that for numerical derivatives of the solution w.r.t parameters may be highly unstable or inaccurate. This is dependent on the numerical properties of the ODE, for eg. stiffness. Recent work has shown that derivatives obtained via automatic differentiation of the solver are more stable and can be used to get accurate derivatives of solutions w.r.t parameters, which can be used for parameter updates, similar to the methods of adjoints **refine and cite**

[1–3]

Hence, a typical calibration exercise requires the user to translate equations into code, connect the code to observation data, define parameter bounds and transformations, select suitable numerical solvers, choose estimation algorithms, and diagnose failures when the fit is poor.

Executing all of the above steps requires expertise and effort to setup the entire estimation process. Any of these steps, if done incorrectly, can cause failure of the estimation problem. Some common failure modes include **include (inappropriate scaling, incorrect optimization setup, insufficient estimation algorithms, poor objective function construction, insufficient constraint flexibility, too broad or too narrow search ranges , inefficient code)**. However, with expert oversight and assistance, this methodology can be reliable at estimating the required parameters.

1.2 Agentic AI workflows

AI agents have recently been adopted across scientific computing for tasks such as geometry and CAD creation [4, 5], solver orchestration [6, 7], end-to-end simulation pipelines and post-processing [8, 9]. These workflows share a characteristic profile: they require substantial manual setup, proceed largely by trial and error, expose a large number of configuration knobs that materially affect the solution yet are easily overlooked, and demand expert knowledge to operate the underlying tools effectively. Crucially, their outcomes are often *verifiable*- the task’s output can be checked against clear evidence, rules, tests, or ground truth rather than judged only by subjective preference. This combination of high manual overhead, expertise requirements, and built-in verifiability makes such workflows particularly amenable to augmentation by AI agents: the agent can shoulder the routine and knowledge-intensive setup, while objective checks bound the cost of any individual mistake. The success of these efforts, together with the recurring need for manual intervention and domain expertise, directly motivates applying AI agents to the parameter estimation problem considered here.

However, delegating complex problems to an AI agent without user oversight risks hallucination, context degradation, and silent, incorrect assumptions, any of which can yield erroneous or unphysical solutions [8, 10]. The risk is compounded by a tendency of even state-of-the-art models toward overconfidence: they frequently present unsupported conclusions with high apparent certainty and are poorly calibrated. An agent left to run unsupervised through a long chain of decisions can therefore accumulate and amplify errors that a domain expert would catch immediately.

The workflow described in this chapter is designed around these constraints. Its aim is to provide a framework that uses AI agents as *skills-driven assistants* for orchestrating parameter estimation problems, rather than as autonomous black boxes. The broader estimation problem is decomposed into a sequence of well-scoped sub-steps, each completed by the agent and each producing intermediate artifacts written to disk. Concretely, the agent is used to identify inconsistencies in the problem specification, repair common input pathologies, transform user-defined model functions into JIT-compilable (just-in-time compiled) routines, and assemble a two-stage optimization pipeline that combines a parallelized global search with gradient-based refinement. At every step the agent surfaces the choices it has made, explains its reasoning, and when a fit is poor recommends concrete changes to improve it.

Claude skills are used to demonstrate the end-to-end workflow. The commands and context files developed here are not tied to that choice, however: the same decomposition, context engineering, and audit-by-intermediate-file strategy transfer in principle to any agent provider the reader prefers.

2 Methods

2.1 Context Assisted Agents via Skills

A transformer language model defines an autoregressive distribution $p(x_t | x_{<t})$ over the next token x_t given all preceding tokens in its context window, repeatedly predicting the next piece of text from everything written so far. This conditioning is implemented by a mechanism called self-attention: as the model processes each token, it scores how relevant every earlier token is to the one currently being processed, using this relevance score to predict the next token. The model has no persistent task-specific memory beyond this: everything that shapes a prediction must be present, as tokens, in the context. All task-relevant behavior is therefore induced at inference time by the context: the instructions, reference material, and examples assembled into the input are the primary levers for controlling an agent.

The context window is also a finite, shared resource, and attention makes its use non-uniform. For each token, the relevance scores described above are normalized so that they sum to one across all the earlier tokens (an operation called a softmax). Adding more tokens to the context window therefore forces the few that actually matter to share this fixed budget with a larger pool of competitors. Loading an entire domain into one static prompt thus scales poorly: it inflates cost and latency, pushes against the length limit, and tends to lower quality as the signal-to-noise ratio of the window falls.

A *skill* exploits these properties directly. Rather than rely on a single static prompt to carry the whole task, a skill is scoped to one sub-task, that determines which material is loaded into the context window precisely when that step executes: the rules the step must follow, the structural templates its output must match, and tool calls the agent may make. Because the model attends over exactly this material, the conventions and constraints written into it propagate into the generated output by construction, while detail belonging to other steps is kept out of the window and out of the attention budget. Assembling context just in time, one step at a time, rather than all at once keeps the active window dense with relevant tokens and well within its length limit. This both sharpens output quality and lets each step be revised independently by editing its context alone,

with no change to the underlying model.

Discuss why API based agents rather than local install: as opposed to local model deployment

2.2 Search Algorithms

2.2.1 Population methods

The framework ships with two population-based global optimizers, Particle Swarm Optimization (PSO) and Differential Evolution (DE), for the coarse search that precedes gradient-based refinement. Both are *gradient-free*, tolerating the stiff, non-smooth losses typical of ODE fitting; *global*, exploring an entire population rather than a single iterate to avoid poor local minima; and *embarrassingly parallel*, since per-candidate evaluations are independent and dispatched in parallel. The population is given a space-filling initialization via an optimized Latin Hypercube (LHS) design [11], and both search in the normalized $[-1, 1]^d$ space so parameters spanning many orders of magnitude are treated on equal footing. The best candidate found seeds the gradient stage.

Particle Swarm Optimization (PSO) The global search defaults to PSO[12], a population-based metaheuristic in which a swarm of N candidate solutions (*particles*) explores the normalized parameter space $[-1, 1]^d$. Particle i carries a position $\mathbf{x}_i \in [-1, 1]^d$ and a velocity $\mathbf{v}_i$. The swarm is seeded with a LHS design to spread the initial positions evenly, and a star topology is used so every particle shares a single global best.

Let $f(\cdot)$ denote the (normalized) loss. Each particle tracks its own best position $\mathbf{p}_i$ and the swarm’s global best $\mathbf{g}$:

$$\mathbf{p}_i \leftarrow \arg \min_{\mathbf{x} \in \{\mathbf{p}_i, \mathbf{x}_i\}} f(\mathbf{x}), \quad \mathbf{g} \leftarrow \arg \min_i f(\mathbf{p}_i). \quad (1)$$

At each iteration k the velocity and position are updated by

$$\mathbf{v}_i^{k+1} = w \mathbf{v}_i^k + c_1 \mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^k) + c_2 \mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^k), \quad (2)$$

$$\mathbf{x}_i^{k+1} = \mathbf{x}_i^k + \mathbf{v}_i^{k+1}, \quad (3)$$

where $\mathbf{r}_1, \mathbf{r}_2 \sim \mathcal{U}(0, 1)^d$ are drawn element-wise, $\odot$ is the Hadamard product, w is the inertia weight, and c_1, c_2 are the cognitive and social coefficients. The velocity is clamped component-wise to $[-0.3, 0.3]$, and particles leaving the box are projected back to the nearest boundary.

Relative to differential evolution, PSO tends to converge faster on easier problem landscapes [cite](#): the velocity memory and shared global attractor let information propagate through the swarm quickly, so good regions are exploited in fewer loss evaluations.

Differential Evolution (DE) As an alternative global search, the framework provides Differential Evolution [13], a population-based evolutionary algorithm well suited to multimodal, non-differentiable objectives. A population of N candidate vectors $\{\mathbf{x}_i\}_{i=1}^N$ in $[-1, 1]^d$ is initialized with LHS and evolved by mutation, crossover, and selection.

For each target vector $\mathbf{x}_i$ at generation k , a *mutant* is formed from the current best and two distinct randomly chosen members ($r_1 \neq r_2 \neq i$):

$$\mathbf{m}_i = \mathbf{x}_{\text{best}}^k + F (\mathbf{x}_{r_1}^k - \mathbf{x}_{r_2}^k), \quad (4)$$

where the differential weight F is sampled per generation from $\mathcal{U}(0.5, 1.0)$ to improve robustness. A trial vector $\mathbf{u}_i$ is built by binomial crossover, mixing the mutant with the target at a per-component rate set by the recombination constant $CR = 0.7$:

$$u_{i,j} = \begin{cases} m_{i,j}, & \text{if } \rho_j \leq CR \text{ or } j = j_{\text{rand}}, \\ x_{i,j}^k, & \text{otherwise,} \end{cases} \quad \rho_j \sim \mathcal{U}(0, 1), \quad (5)$$

where j_{rand} guarantees at least one component is taken from the mutant. Greedy selection retains whichever of the trial or target has lower loss:

$$\mathbf{x}_i^{k+1} = \begin{cases} \mathbf{u}_i, & \text{if } f(\mathbf{u}_i) \leq f(\mathbf{x}_i^k), \\ \mathbf{x}_i^k, & \text{otherwise.} \end{cases} \quad (6)$$

Relative to PSO, DE is more robust on highly multimodal or ill-conditioned landscapes[13]: its difference-vector mutation adapts the search scale to the population’s current spread, and the greedy, elitist selection prevents the population from drifting away from good solutions. It is therefore the safer choice when PSO stalls in a poor local minimum, at the cost of more loss evaluations to converge.

2.2.2 Gradient-based methods

The best candidate from the global search is refined by a gradient-based optimizer that minimizes the same normalized loss $f(\mathbf{x})$ over $[-1, 1]^d$. Gradients are obtained by *automatic differentiation* (AD) through the full discretized ODE solve, rather than by finite differences. Because AD differentiates the discretized solver–objective map itself, it yields exact discrete sensitivities and requires no perturbation step size—a quantity that is notoriously hard to choose for stiff or highly nonlinear systems [14, 15]. **include a parallel to discrete adjoints** The update is a projected step that keeps iterates inside the search box,

$$\mathbf{x}^{k+1} = \Pi_{[-1,1]^d}(\mathbf{x}^k + \Delta\mathbf{x}^k), \quad \Delta\mathbf{x}^k = \mathcal{U}(\nabla f(\mathbf{x}^k), \mathbf{x}^k), \quad (7)$$

where Π is clipping onto the bounds and $\mathcal{U}$ is the optimizer’s update rule. The framework exposes both a first-order method (Adam, with an exponentially decaying learning-rate schedule) and a second-order quasi-Newton method (L-BFGS); the best iterate is tracked throughout and returned, and a failed integration is caught so the search exits gracefully with the best solution so far.

This stage is intended for *fine-tuning* rather than global discovery: it converges quickly but needs a good initial guess, which the population-based search provides. It is resource-efficient, optimizing a single iterate instead of a whole population, and scales well with dimension, since AD computes the full gradient at a cost comparable to a single loss evaluation regardless of the number of parameters—unlike finite differences, whose cost grows linearly with dimension.

The gradient based search is also the more fragile of the two methods, and several failure modes are worth noting. The computed gradient inherits the accuracy of the forward solve, so loosely converged or under-resolved integration injects noise into ∇f and can cause the iterates to diverge; the solver tolerances must generally be tighter for the gradient stage than for the global search to allow for a stable adjoint computation. Excessively noisy data has a similar effect, producing a rugged loss surface whose local gradients point away from the true optimum. Some

systems are ill-suited to differentiation: non-smooth dynamics (event triggers, clipping, piecewise or discontinuous right-hand sides) break differentiability. Even for smooth, well-behaved systems, convergence can stall on *sloppy* or ill-conditioned problems where the loss has long flat valleys (near-degenerate parameter combinations), and a poor initial guess can leave the optimizer in the wrong basin since the local search cannot escape it. Finally, stiff systems may drive the adaptive solver to exceed its step budget, causing an integration failure mid-search; the framework guards against this by tracking the best iterate and exiting gracefully when a solve fails.

2.3 Post-Fit Sloppiness Diagnostic

After the gradient stage converges, the framework automatically characterizes how well the data constrain the fit by computing the local curvature of the loss at the optimum—the *sloppiness* spectrum in the sense of Gutenkunst *et al.* and Hass *et al.* [16, 17]. The analysis is performed in the coordinate the search actually uses: let $\mathbf{u}$ denote the per-parameter axis with $u_i = \log_{10} \theta_i$ for parameters flagged LOGSCALE and $u_i = \theta_i$ otherwise, so curvature in a log-scaled parameter is measured per decade. Writing the converged best fit as $\mathbf{u}^*$, the framework forms the Hessian of the normalized loss,

$$H = \nabla_{\mathbf{u}}^2 f(\mathbf{u}^*) \in \mathbb{R}^{d \times d}, \quad (8)$$

by second-order automatic differentiation through the entire ODE solve, falling back to a symmetric finite difference of the (first-order AD) gradient when higher-order differentiation through the stiff implicit solver is unavailable. The Hessian is symmetrized, $H \leftarrow \frac{1}{2}(H + H^\top)$, and diagonalized,

$$H = V \Lambda V^\top, \quad \Lambda = \text{diag}(\lambda_1 \geq \dots \geq \lambda_d), \quad (9)$$

yielding an eigenvalue spectrum $\{\lambda_k\}$ and orthonormal eigenvectors $\{\mathbf{v}_k\}$. Each eigenvector is a *direction* in parameter space (a linear combination of the u_i) and its eigenvalue is the curvature of the loss along that direction: large λ_k marks a *stiff*, well-constrained combination, small λ_k a *sloppy*, poorly-constrained one, and $\lambda_k \rightarrow 0$ a flat, practically non-identifiable direction.

The reported verdict uses the eigenvalues normalized by the largest, $\hat{\lambda}_k = \lambda_k / \lambda_1$. A direction is flagged non-identifiable when $\hat{\lambda}_k$ falls below a floor $\varepsilon_0 = 10^{-6}$, and the fit is termed *sloppy* when the identifiable modes span more than six orders of magnitude,

$$\log_{10} \left(\frac{\lambda_1}{\lambda_{\min}^{\text{ident}}} \right) > 6 \quad \text{or} \quad \exists k : \hat{\lambda}_k < \varepsilon_0, \quad (10)$$

where $\lambda_{\min}^{\text{ident}}$ is the smallest eigenvalue above the floor.

This is a local, quadratic measure valid only at a converged optimum, so it detects the flat valleys that stall gradient descent but does not, by itself, map the global extent of a non-identifiable valley, i.e., it is a measure of the quality of the current fit, and not whether there may be a better global minimum at some other solution.

3 Procedure: fitting an equation system end to end

This section describes the steps for running a fitting problem end to end. The framework can be used to fit a set of ODE equations to one or more datasets simultaneously. The overall fitting

process is divided into 5 steps. Four of them are executed using agent skills that read and write the session's files; the fifth step is an ordinary Python run. During every skill execution, the agent will interactively gather information and construct the problem from information provided by the user. The 5 steps can be visualized as

`/pfit-new → /pfit-check → /pfit-jax → /pfit-run → /pfit-diagnose`

The subsequent sections describe setup considerations, before walking through skill execution and the artifacts generated by the program while running the estimation.

Before starting

It is recommended that the user install the required libraries in a local virtual environment. it is **highly** recommended to stick to the pinned versions given in the `requirements.txt` file to avoid performance regressions caused by other versions.

To fit a new system, a user must create `sessions/<name>/inputs/` and put the required data there. There are generally five things required to run a fitting, described in [Table 1](#) below:

Artifact	Description
Equation System	System of ODEs to fit
Dataset(s) (CSV)	The dataset to fit to
Initial conditions	The initial condition corresponding to each dataset
Search ranges	Search ranges for each parameter - suggested if not provided
Loss formulation	Loss construction using ODE solution - suggested if not provided

[provide an example here](#)

Fitting systems described in papers: If attempting to reconstruct a system from a paper, it is possible to pass the paper to the `/pfit-new` skill, and point to the relevant sections to obtain some or all of the above information.

Consistent Units: It is important to remember that units need to be consistent between the dataset, equation system, initial conditions, and search ranges. Unless very obvious, the framework will not perform unit transformations and this will lead to an incorrect answer.

Dataset CSV column headers: It is very useful to have headers for each column of the dataset CSV(s), describing what each column represents, along with its units.

Requirements on the dataset files

The data file(s) is never modified by the framework, so it has to arrive in a form the loader and the solver can both consume. The requirements in [Table 2](#) outline the necessary structure the file must follow for the framework to use it as intended, and [Table 3](#) highlights features of the framework on how different **data faults** are handled.

[add table label, increase spacing](#)

Requirement	Remarks
Plain comma-separated text	Other separators eg. tab-delimiters do not load
Rectangular, i.e every row has the same number of fields	The structure must be rectangular, even if some observables are not sampled at every recorded time.
At least two rows and two columns	At least 2 datapoints (excluding initial conditions) must be present
No trailing comma at the end of a row	Data is improperly read
Column 0 is always time	The recorded times are the grids at which integrated ODE solutions are saved
Time must be <i>strictly</i> increasing	A non-increasing sequence in time will raise an error
Cell NaN values	Missing values and text becomes NaN, which is masked
Blank cells where a measurement is genuinely missing	Supported by masking resulting NAN values
Different datasets measure different outputs across experiments	Every dataset file must have the same columns. If different datasets measure different outputs, the solution is to have the union of all columns present in each CSV, and leave the missing entries blank, which will be dealt with using the masking strategy discussed before
Initial conditions	If initial conditions are part of the dataset, the user ensures that they match initial conditions provided outside of the data in the problem definition

Fitting multiple datasets simultaneously add example of file labeling for datasets

Fitting several records at once means looking for one parameter set that explains all of them. That is the correct approach when they are the same system measured under different conditions—different initial conditions, applied forcing functions, or repeat runs of one protocol. If the records are intended to have different parameters, those should be separate sessions, fitted independently and compared afterwards.

To facilitate fitting against multiple dataset files, every data CSV should exist in the session’s input directory, and the problem setup statement should describe what differed between them. For example:

Six runs of the same experiment, all in inputs/. Same protocol and the same columns in every file. What changed between them is the starting calcium: 0.1, 0.2, 0.4, 1, 3 and 10 micromolar, in that file order.

The agent may ask follow up questions to ensure correctness. It is important to be explicit about what differs between datasets to ensure correctness.

The total loss is then evaluated once per dataset file (or record), against that record’s own data, time grid and initial conditions, and the results are averaged over records. Each record gets its own output file, numbered in the order initially listed (give an example this). The records can be treated as completely independent in terms of experiment; however they must share the meaning and order of their columns (see Table 2), since that mapping is positional in the setup code.

Some necessary things to remember for fitting multiple datasets:

The average is *unweighted*. A record whose residuals are numerically larger will dominate the fit, so records of very different magnitude need a loss that normalises each against its own scale.

You see only that average while the fit runs. Per-record quality is not reported separately, so a fit that is excellent on five records and useless on the sixth reads as merely mediocre. Plot the per-record results before concluding anything. And if your files start at different times, leave the start time alone. Left unset, each record begins at its own first timestamp, which is almost always what you want. Set explicitly, it applies to every record at once, and any file whose data begins earlier will be rejected.

Objective function

The construction of the objective is central to the quality of the obtained estimates. The objective **must** capture all constraints on the ODE solution the user requires. Generally, for an ODE of the form $\mathbf{y} = ODE(\mathbf{x})$, the general form of a constraint is $f_i(\mathbf{x}, \mathbf{y}) \leq 0$ [add eq numbers etc](#) and this is added to the loss through a weighted linear combination of penalty functions of the form. [add loss function form, add a note on relative differentiability of different kinds of losses](#)

This requires some user expertise and insight, as an underconstrained problem implies the solution may have unanticipated or undesired dynamics, and an overconstrained problem runs the risk of an optimizer not finding a feasible solution during the global optimization stage. It should be noted that the user may choose to allow the agent to formulate the loss itself based on the problem specification. This works for simpler problems, but for more complex problems, it is advised the user specify how the loss is constructed from the ODE solution manually. Broadly, there are best practices for constructing the objective function, listed in [Table](#).

Best Practice	Remarks
Every term of the overall objective should be of similar scale in magnitude (preferably close to unity)	It is recommended to divide each objective term by a scale belonging to its observable, like its own peak, value range, or measurement uncertainty. This enables more uniform estimation of parameters, and a relative importance of terms can be controlled by setting channel weights. Several assumptions in the framework are built around this scale being close to unity. link example
Weighting of samples within a trajectory	add ARC example Some trajectories to fit to might only carry relevant dynamics in small subsets of the overall time history, but by default each sample in time history may be weighted equally. This can lead to poor estimates, and the loss formulation can be weighted unequally in time to mitigate this
Scaling of value <i>within</i> a loss term	In some instances, the data being estimated can span several orders of magnitude within a single channel and trajectory. Users are advised to use logarithmic scalings to allow for uniform contributions to the objective in such cases
Differentiability of constraints	To allow for gradient-based fine tuning, it is recommended to add penalty terms in a form that allow for differentiability. For example, rather than adding a step function add forms the user can use a sigmoid with high weight

[add table label, increase spacing](#)

Fitting multiple datasets using a single model A few additional guidelines exist when fitting a model to multiple datasets:

- The same loss function is applied to each dataset, so it has to be written to work for any of them.
- Because the datasets are averaged with equal weight, each must be normalized against its own scale rather than a constant shared by the session.
- It is recommended to average the estimate of a single dataset rather than sum, to avoid disproportionate contribution by any one term

3.1 Starting a new session

A new fitting problem begins with

```
/pfit-new <session-name>
```

This command converts a user description of a scientific fitting problem into a session specification. The user provides the model equations, data files, parameter bounds or fixed values, initial conditions, forcing functions, and the loss definition. The agent's role is to assemble these inputs into the two files used by the framework, `inputs/user_input.yaml` and `generated/user_model.py`.

Before writing those files, the agent must have a complete specification of the run: the ODE states and their ordering, the trainable and fixed parameters, the experiments to be fitted, the initial condition and forcing history for each experiment, the mapping from data columns to observables or inputs, and the loss used to compare model outputs with measurements. If the user's note is not complete, the agent asks only for the missing decisions and declarations.

3.1.1 User-provided problem setup

A primary goal of this framework is to make it as easy as possible for a user to provide a problem description. The user is not expected to write YAML or Python. As discussed before, the user may also point at the relevant sections of a research paper while running this skill, and the agent will perform setup based on that. However, a user may also provide the problem setup in an arbitrary format.

Shown below is an example input format a user may provide. This example covers the case of a user providing multiple datasets, with each dataset having different observables, different rates of sampling, and how a user may provide this information and steps they would have to take. This is intended as a sample problem only.

```
# Drug response fitting problem

## User model
files = [pulse_low_raw.csv, washout_raw.csv]
states = [D, R]

Initial conditions:
    pulse_low_raw.csv: D = 0, R = 0
```

```
washout_raw.csv:   D = 0, R = 0.3
```

ODE:

```
dD/dt = dose_rate(t) - k_elim D
dR/dt = k_on D (1 - R) - k_off R
```

Observables:

```
signal_model(t) = baseline + gain R(t)
recovery_model(t) = R(t)
```

Trainable and fixed parameters

```
k_elim:   0.01   to 10,   log scale
k_on:     0.001  to 10,   log scale
k_off:    0.001  to 10,   log scale
baseline: 50     to 150,  linear scale
gain:     1      to 200,  linear scale
fixed parameters: none
```

Loss function calculation using model outputs and datasets

pulse_low_raw.csv:

```
t = time_min
dose_rate(t) = dose_rate
signal_obs = signal
sigma = signal_sd
```

washout_raw.csv:

```
t = time_min
dose_rate(t) = 0
recovery_obs = recovery
sigma = recovery_sd
```

For each finite measured value:

```
r_signal = (signal_model(t_k) - signal_obs,k) / signal_sd,k
r_recovery = (recovery_model(t_k) - recovery_obs,k) / recovery_sd,k
loss = sqrt(mean(r^2)) over all finite residuals
```

Blank measured entries are missing measurements and are excluded.

This form states the model, unknowns, data mapping, and objective in the terms a scientist naturally uses, while still giving the agent enough information to construct the internal schema.

3.1.2 Preparing the data files

Raw data files may contain different measured quantities. In a multi-experiment fit, however, the framework expects every fitting CSV to have the same column layout: the time column first,

followed by the same forcing, measurement, and uncertainty columns in the same order. When an observable is absent from one experiment, the user keeps the column and leaves its entries blank. The agent is not allowed to modify data files or write derived copies; it identifies the required layout, and the user creates the framework-ready CSV files.

The following raw files already use consistent names and time units, but they measure different outputs. The pulse experiment measures `signal`; the washout experiment measures `recovery`. Both outputs enter the loss, so the canonical layout must contain the union of these measured channels.

<code>pulse_low_raw.csv</code>	<code>washout_raw.csv</code>
<code>time_min,dose_rate,signal,signal_sd</code>	<code>time_min,dose_rate,recovery,recovery_sd</code>
<code>0,0.0,100.0,4.0</code>	<code>0,0.0,0.82,0.04</code>
<code>1,1.0,,4.2</code>	<code>0.5,0.0,0.78,0.04</code>
<code>2,1.0,113.5,4.4</code>	<code>1.0,0.0,,0.05</code>
<code>4,0.0,,4.1</code>	<code>2.5,0.0,0.69,0.04</code>
<code>6,0.0,104.5,4.0</code>	<code>5.0,0.0,0.58,0.03</code>
	<code>8.0,0.0,0.51,0.03</code>

For fitting, the user prepares both files with the common column layout

```
time_min,dose_rate,signal,signal_sd,recovery,recovery_sd
```

The pulse file has no recovery measurements, so the recovery columns are blank. The washout file has no signal measurements, so the signal columns are blank. Blank measured values are loaded as NaN and excluded by an automatically masked loss. The time grids may differ between files, but the time unit must already be consistent; the framework does not infer or perform unit conversions. So, the user should modify each raw dataset file as shown below. Note the column layout.

<code>pulse_low.csv</code>	<code>washout.csv</code>
<code>time_min,dose_rate,signal,signal_sd,</code>	<code>time_min,dose_rate,signal,signal_sd,</code>
<code>recovery,recovery_sd</code>	<code>recovery,recovery_sd</code>
<code>0,0.0,100.0,4.0,,</code>	<code>0,0.0,,,0.82,0.04</code>
<code>1,1.0,,4.2,,</code>	<code>0.5,0.0,,,0.78,0.04</code>
<code>2,1.0,113.5,4.4,,</code>	<code>1.0,0.0,,,0.05</code>
<code>4,0.0,,4.1,,</code>	<code>2.5,0.0,,,0.69,0.04</code>
<code>6,0.0,104.5,4.0,,</code>	<code>5.0,0.0,,,0.58,0.03</code>
	<code>8.0,0.0,,,0.51,0.03</code>

The same loss formulation runs once per experiment, and NaN masks determine which terms are active for that experiment. The framework automates the process of ensuring losses do not go to NaN due to division by zero, but where possible the user prompt should explicitly mention this possibility.

3.1.3 Agent review and generated files

From the note and CSV headers, the agent infers that the two experiments share one ODE system and one trainable parameter vector, but differ in the initial condition for R and in their forcing histories. The measured columns are compared to calculated observables: `signal` to `signal_model`

= baseline + gain R, and recovery to recovery_model = R. The corresponding uncertainty columns weight the residuals, and blank measurements require a finite-data mask.

If information is missing, the agent asks targeted questions such as:

The recovery column appears only in the washout file. Should it measure $R(t)$?

The signal column is absent from the washout file. Should it be blank there and masked?

Should dose_rate(t) be zero throughout the washout experiment?

Once the specification is complete, /pfit-new writes the session files. The YAML records the user-prepared CSV files, initial conditions, parameter bounds, observables, and column declarations. The model file records the ODE right-hand side, forcing interpolation, calculated observables, and the masked uncertainty-weighted loss over both measured channels. The user then reviews the generated state ordering, parameter ranges, equations, and data-column usage.

3.2 Checking the session

After creating a session, the user runs

```
/pfit-check <session-name>
```

This step verifies that the problem is ready for translation to JAX and fitting. It reads the generated configuration, the generated model file, and the CSV files named in the configuration. It serves as a wider net to catch errors in the problem specification before numerical optimization begins.

The procedure checks the implementation by answering four questions:

1. Are the data files usable by the framework: rectangular CSV files, valid time columns, expected column names, and compatible missing-value conventions?
2. Is the model specification self-consistent: declared states, parameters, initial conditions, observables, and data columns all refer to the same objects?
3. Is the loss well defined for the available measurements, including blank or NaN entries?
4. Are the requested numerical settings adequate for the scale and structure of the problem?

The agent may revise the generated YAML or Python files when the correction is a mechanical consequence of the specification. The report is written to

```
sessions/<session-name>/generated/user_input_check.txt
```

and separates findings into critical errors, warnings, and recommendations. Critical errors are expected to prevent translation or a valid run and must be resolved before continuing. Warnings identify choices that may make the fit less reliable or harder to interpret. Recommendations are generated from the framework's documented checking rules and must be supported by evidence from the current session, such as parameter ranges, data scales, missing values, solver settings, or loss construction. Examples include increasing the global search budget, tightening solver tolerances for small state variables, or normalizing residuals across measured channels. if not done already.

3. Translation to JAX

```
\texttt{/pfit-jax \textless session\textgreater{}}
```

A mechanical step that translates the final model into the JIT-compilable script, and verifies that the result imports.

3.3 Running the estimation

After the JAX program has been generated, the user starts the parameter estimation with

```
/pfit-run <session-name>
```

By default, this command performs the full fitting procedure: a global search followed by local gradient-based refinement. A refinement-only run can be requested when a previous run has already found a suitable design point:

```
/pfit-run <session-name> gradient-only
```

The command starts a gradient-only run, given that the seed design point is available from the previous stage.

Each run is written to a new output directory, so earlier results are preserved. At launch, the agent reports the exact run directory and a local live-view URL. **add an image of the live fit caption: The live view shows the active optimization stage, current best loss, elapsed time, and current best parameter values**

If progress is very slow, the agent reports the evidence from the active run and may propose an intervention, such as stopping the current run, preserving its artifacts, changing a numerical setting, regenerating the JAX program, and starting a new run. Such an intervention is a restart and is applied only after the user approves the specific change.

When the run finishes, a completion report including the final objective value, fitted parameter values, and the locations of the saved result tables is written. The agent also generates and inspects measured-versus-fitted plots for every fitted observable and experiment.

The user should review the reported parameters and saved fit plots before drawing conclusions. Formal assessment of convergence, residual structure, model mis-specification, and practical identifiability is deferred to the next workflow step.

3.4 Diagnosing a completed fit

After a fitting run has finished, the user requests a post-fit assessment with

```
/pfit-diagnose <session-name> <run-id>
```

The diagnosis step evaluates a completed run. The agent reads the configuration and model snapshots saved with that run, and bases the assessment on the artifacts written by the optimizer.

The purpose of this step is to decide the quality of the fit. The agent examines the global-search and refinement logs, the final parameter values, the sloppiness report (discussed in [Section](#)), the saved trajectories, residuals, and the measured-versus-fitted plots. The report is written to

sessions/<session-name>/outputs/<run-id>/fit_diagnosis.txt

Typical findings include failed integrations, insufficient global-search budget, premature population collapse, a gradient stage that stopped before convergence, parameters pinned at bounds, and poorly scaled residuals.

Recommendations are proposed as edits to the working session. The user chooses which recommendations to apply. Changes to the equations, bounds, log scaling, solver family, or population-search settings generally require the checked translation and full fitting workflow to be repeated, but the user may also run a gradient-only refinement from the diagnosed design point.

4 Examples

A comprehensive list of examples is available at the [repo](#)

The focus is going to be on problems with experimentally gathered data. Will demo a range of systems to fit to: highly nonlinear, stiff systems from other fields, to multi-dataset examples from computational biology. All examples done on experimental data; so the model is not a perfect representation of the underlying dynamics.

A few examples from biological sciences are taken directly from an external, community-curated benchmark collection (the PEstab benchmark suite [17]). It should be noted that the loss formulations in the following problem can differ from those used in the benchmarks, in part because the framework currently **computes point estimate models and does not directly estimate measurement noise**.

5 Summary and Conclusions

A note on model appropriateness

The framework finds a good solution based on equations, data, losses and constraints describe the problem. One of the main purposs of the framework is to allow users to easily fit to noisy, real world data. In most applications, this means that the model is an approximate representation of real world dynamics, and the framework cannot guarantee perfect fits.

Competing Interests

The author has no conflicts of interest to declare that are relevant to the content of this chapter.

References

References

- [1] Saakaar Bhatnagar. An agentic ai workflow to simplify parameter estimation of complex differential equation systems, 2025. URL <https://arxiv.org/abs/2509.07283>.

- [2] Andreas Raue, Clemens Kreutz, Thomas Maiwald, Julie Bachmann, Marcel Schilling, Ursula Klingmuller, and Jens Timmer. Structural and practical identifiability analysis of partially observed dynamical models by exploiting the profile likelihood. *Bioinformatics*, 25(15): 1923–1929, 2009. doi: 10.1093/bioinformatics/btp358.
- [3] Ryan N. Gutenkunst, Joshua J. Waterfall, Fergal P. Casey, Kevin S. Brown, Christopher R. Myers, and James P. Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS Computational Biology*, 3(10):e189, 2007. doi: 10.1371/journal.pcbi.0030189.
- [4] Mohammad S Khan, Sankalp Sinha, Talha U Sheikh, Didier Stricker, Sk A Ali, and Muhammad Z Afzal. Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts. *Advances in Neural Information Processing Systems*, 37:7552–7579, 2024.
- [5] Yandong Guan, Xilin Wang, Ximing Xing, Jing Zhang, Dong Xu, and Qian Yu. Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward. *Advances in Neural Information Processing Systems*, 38:59765–59789, 2026.
- [6] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam: an llm-based multi-agent framework for cfd. *arXiv preprint arXiv:2407.21320*, 2024.
- [7] Ling Yue, Peijing Xu, Tingwen Zhang, Nithin Somasekharan, and Shaowu Pan. Foamgpt: Fine-tuning large language model for agentic automation of cfd simulations with openfoam. *Advances in Neural Information Processing Systems*, 2025.
- [8] Zhaoyue Xu, Long Wang, Chunyu Wang, Yixin Chen, Qingyong Luo, Hua-Dong Yao, Shizhao Wang, and Guowei He. Cfdagent: A language-guided, zero-shot multi-agent system for complex flow simulation. *Physics of Fluids*, 37(11), 2025.
- [9] Yuxuan Chen, Xu Zhu, Hua Zhou, and Zhuyin Ren. Metaopenfoam 2.0: Large language model driven chain of thought for automating cfd simulation and post-processing. *arXiv preprint arXiv:2502.00498*, 2025.
- [10] Donggeun Park, Hyeonbin Moon, and Seunghwa Ryu. A self-correcting multi-agent llm framework for language-based physics simulation and explanation. *npj Artificial Intelligence*, 2, 2026. doi: 10.1038/s44387-025-00057-z.
- [11] Guillaume Damblin, Mathieu Couplet, and Bertrand Iooss. Numerical studies of space filling designs: Optimization of latin hypercube samples and subprojection properties. *Journal of Simulation*, 7(4):276–289, 2013.
- [12] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN’95 - International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi: 10.1109/ICNN.1995.488968.
- [13] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.

- [14] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18(153):1–43, 2018.
- [15] Yingbo Ma, Vaibhav Dixit, Michael J Innes, Xingjian Guo, and Chris Rackauckas. A comparison of automatic differentiation and continuous sensitivity analysis for derivatives of differential equation solutions. In *2021 IEEE high performance extreme computing conference (HPEC)*, pages 1–9. IEEE, 2021.
- [16] Ryan N Gutenkunst, Joshua J Waterfall, Fergal P Casey, Kevin S Brown, Christopher R Myers, and James P Sethna. Universally sloppy parameter sensitivities in systems biology models. *PLoS computational biology*, 3(10):e189, 2007.
- [17] Helge Hass, Carolin Loos, Elba Raimúndez-Álvarez, Jens Timmer, Jan Hasenauer, and Clemens Kreutz. Benchmark problems for dynamic modeling of intracellular processes. *Bioinformatics*, 35(17):3073–3082, 2019.

Tables

Table 1: Suggested input files for the agentic parameter estimation workflow.

File	Purpose
<code>problem.xml</code>	Structured definition of states, parameters, data, and settings
<code>model.py</code>	User-editable model and observation functions
<code>data.csv</code>	Observation data and time points
<code>config.yaml</code>	Solver, optimizer, output, and reproducibility settings

Figure Legends

Fig. 1. End-to-end agentic parameter estimation workflow. The user supplies a structured problem definition and a Python model skeleton. The agentic workflow validates the inputs, generates a differentiable calibration pipeline, runs global exploration and gradient-based refinement, and exports fitted parameters, diagnostics, and reproducible code.

Fig. 2. Example structure of the problem specification file. The schema records state variables, parameters, bounds, initial conditions, observation mappings, data paths, solver settings, and optimizer settings.

Fig. 3. Two-stage optimization strategy. A global search explores the bounded parameter space and identifies candidate regions, after which gradient-based refinement improves the best candidates using differentiable model evaluations.

Fig. 4. Example diagnostic outputs from a fitted ODE model. Suggested panels include model trajectories against observed data, residuals over time, fitted parameter values relative to bounds, and convergence history.